Advanced Data Structures

Binary Search Trees, Python Generators, and Hash Tables

Martin Benning

University College London
ENGF0034 – Design and Professional Skills

27 October 2026



Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Contrast linear and hierarchical search trade-offs, and state the formal BST invariant.
2. Implement recursive BST lookup, insertion, and three deletion cases (including Hibbard value swapping).
3. Explain lazy evaluation in Python generators (`yield`, `yield from`) and analyse auxiliary memory.
4. Derive BST height bounds, identify degenerate worst cases, and motivate self-balancing trees.
5. Design hash tables with separate chaining, analyse expected $\mathcal{O}(1)$ lookup, and apply hash sets to coordinate grids.

Lecture Roadmap

The Search Problem & Tree Foundations
Binary Search Trees & Invariants
Tree Traversals & Python Generators
Binary Search Tree Deletion
BST Complexity and Tree Balance
Hash Tables and Collision Resolution
Summary and Outlook

Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Contrast linear and hierarchical search trade-offs, and state the formal BST invariant.
2. Implement recursive BST lookup, insertion, and three deletion cases (including Hibbard value swapping).
3. Explain lazy evaluation in Python generators (`yield`, `yield from`) and analyse auxiliary memory.
4. Derive BST height bounds, identify degenerate worst cases, and motivate self-balancing trees.
5. Design hash tables with separate chaining, analyse expected $\mathcal{O}(1)$ lookup, and apply hash sets to coordinate grids.

Lecture Roadmap

The Search Problem & Tree Foundations
Binary Search Trees & Invariants
Tree Traversals & Python Generators
Binary Search Tree Deletion
BST Complexity and Tree Balance
Hash Tables and Collision Resolution
Summary and Outlook

Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Contrast linear and hierarchical search trade-offs, and state the formal BST invariant.
2. Implement recursive BST lookup, insertion, and three deletion cases (including Hibbard value swapping).
3. Explain lazy evaluation in Python generators (`yield`, `yield from`) and analyse auxiliary memory.
4. Derive BST height bounds, identify degenerate worst cases, and motivate self-balancing trees.
5. Design hash tables with separate chaining, analyse expected $\mathcal{O}(1)$ lookup, and apply hash sets to coordinate grids.

Lecture Roadmap

The Search Problem & Tree Foundations
Binary Search Trees & Invariants
Tree Traversals & Python Generators
Binary Search Tree Deletion
BST Complexity and Tree Balance
Hash Tables and Collision Resolution
Summary and Outlook

Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Contrast linear and hierarchical search trade-offs, and state the formal BST invariant.
2. Implement recursive BST lookup, insertion, and three deletion cases (including Hibbard value swapping).
3. Explain lazy evaluation in Python generators (`yield`, `yield from`) and analyse auxiliary memory.
4. Derive BST height bounds, identify degenerate worst cases, and motivate self-balancing trees.
5. Design hash tables with separate chaining, analyse expected $\mathcal{O}(1)$ lookup, and apply hash sets to coordinate grids.

Lecture Roadmap

The Search Problem & Tree Foundations
Binary Search Trees & Invariants
Tree Traversals & Python Generators
Binary Search Tree Deletion
BST Complexity and Tree Balance
Hash Tables and Collision Resolution
Summary and Outlook

Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Contrast linear and hierarchical search trade-offs, and state the formal BST invariant.
2. Implement recursive BST lookup, insertion, and three deletion cases (including Hibbard value swapping).
3. Explain lazy evaluation in Python generators (`yield`, `yield from`) and analyse auxiliary memory.
4. Derive BST height bounds, identify degenerate worst cases, and motivate self-balancing trees.
5. Design hash tables with separate chaining, analyse expected $\mathcal{O}(1)$ lookup, and apply hash sets to coordinate grids.

Lecture Roadmap

The Search Problem & Tree Foundations
Binary Search Trees & Invariants
Tree Traversals & Python Generators
Binary Search Tree Deletion
BST Complexity and Tree Balance
Hash Tables and Collision Resolution
Summary and Outlook

Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Contrast linear and hierarchical search trade-offs, and state the formal BST invariant.
2. Implement recursive BST lookup, insertion, and three deletion cases (including Hibbard value swapping).
3. Explain lazy evaluation in Python generators (`yield`, `yield from`) and analyse auxiliary memory.
4. Derive BST height bounds, identify degenerate worst cases, and motivate self-balancing trees.
5. Design hash tables with separate chaining, analyse expected $\mathcal{O}(1)$ lookup, and apply hash sets to coordinate grids.

Lecture Roadmap

The Search Problem & Tree Foundations
Binary Search Trees & Invariants
Tree Traversals & Python Generators
Binary Search Tree Deletion
BST Complexity and Tree Balance
Hash Tables and Collision Resolution
Summary and Outlook

The Search Problem & Tree Foundations

The Search Problem: Contiguous vs Linked Trade-offs

Revisiting Weeks 2 and 3

Linear structures force a performance compromise:

- **Unsorted Array:** $\mathcal{O}(1)^{\mathrm{A}}$ (amortised) append, but $\mathcal{O}(N)$ linear scan.
- **Sorted Array:** $\mathcal{O}(\log N)$ binary search, but $\mathcal{O}(N)$ shifts on insert and delete.
- **Linked List:** $\mathcal{O}(1)$ insert at a known pointer, but $\mathcal{O}(N)$ search without pointer.

The Core Dilemma

Can we achieve $\mathcal{O}(\log N)$ search without paying $\mathcal{O}(N)$ shifts on insertion?

Comparison of Linear Structures

Structure	Search	Insert	Delete
Unsorted Array	$\mathcal{O}(N)$	$\mathcal{O}(1)^{\mathrm{A}}$	$\mathcal{O}(N)$
Sorted Array	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$
Linked List	$\mathcal{O}(N)$	$\mathcal{O}(1)$	$\mathcal{O}(N)$
Target (BST)	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$

^{A} Amortised time; linked list $\mathcal{O}(1)$ insert assumes a given node pointer.

The Solution Concept

Organise nodes into a **tree hierarchy** that halves the remaining search space at every step.

The Search Problem: Contiguous vs Linked Trade-offs

Revisiting Weeks 2 and 3

Linear structures force a performance compromise:

- **Unsorted Array:** $\mathcal{O}(1)^{A}$ (amortised) append, but $\mathcal{O}(N)$ linear scan.
- **Sorted Array:** $\mathcal{O}(\log N)$ binary search, but $\mathcal{O}(N)$ shifts on insert and delete.
- **Linked List:** $\mathcal{O}(1)$ insert at a known pointer, but $\mathcal{O}(N)$ search without pointer.

The Core Dilemma

Can we achieve $\mathcal{O}(\log N)$ search without paying $\mathcal{O}(N)$ shifts on insertion?

Comparison of Linear Structures

Structure	Search	Insert	Delete
Unsorted Array	$\mathcal{O}(N)$	$\mathcal{O}(1)^{A}$	$\mathcal{O}(N)$
Sorted Array	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$
Linked List	$\mathcal{O}(N)$	$\mathcal{O}(1)$	$\mathcal{O}(N)$
Target (BST)	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$

^{A} Amortised time; linked list $\mathcal{O}(1)$ insert assumes a given node pointer.

The Solution Concept

Organise nodes into a **tree hierarchy** that halves the remaining search space at every step.

The Search Problem: Contiguous vs Linked Trade-offs

Revisiting Weeks 2 and 3

Linear structures force a performance compromise:

- **Unsorted Array:** $\mathcal{O}(1)^{\mathrm{A}}$ (amortised) append, but $\mathcal{O}(N)$ linear scan.
- **Sorted Array:** $\mathcal{O}(\log N)$ binary search, but $\mathcal{O}(N)$ shifts on insert and delete.
- **Linked List:** $\mathcal{O}(1)$ insert at a known pointer, but $\mathcal{O}(N)$ search without pointer.

The Core Dilemma

Can we achieve $\mathcal{O}(\log N)$ search without paying $\mathcal{O}(N)$ shifts on insertion?

Comparison of Linear Structures

Structure	Search	Insert	Delete
Unsorted Array	$\mathcal{O}(N)$	$\mathcal{O}(1)^{\mathrm{A}}$	$\mathcal{O}(N)$
Sorted Array	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$
Linked List	$\mathcal{O}(N)$	$\mathcal{O}(1)$	$\mathcal{O}(N)$
Target (BST)	$\mathcal{O}(\log \mathbf{N})$	$\mathcal{O}(\log \mathbf{N})$	$\mathcal{O}(\log \mathbf{N})$

^{A} Amortised time; linked list $\mathcal{O}(1)$ insert assumes a given node pointer.

The Solution Concept

Organise nodes into a **tree hierarchy** that halves the remaining search space at every step.

The Search Problem: Contiguous vs Linked Trade-offs

Revisiting Weeks 2 and 3

Linear structures force a performance compromise:

- **Unsorted Array:** $\mathcal{O}(1)^{\mathcal{A}}$ (amortised) append, but $\mathcal{O}(N)$ linear scan.
- **Sorted Array:** $\mathcal{O}(\log N)$ binary search, but $\mathcal{O}(N)$ shifts on insert and delete.
- **Linked List:** $\mathcal{O}(1)$ insert at a known pointer, but $\mathcal{O}(N)$ search without pointer.

The Core Dilemma

Can we achieve $\mathcal{O}(\log N)$ search without paying $\mathcal{O}(N)$ shifts on insertion?

Comparison of Linear Structures

Structure	Search	Insert	Delete
Unsorted Array	$\mathcal{O}(N)$	$\mathcal{O}(1)^{\mathcal{A}}$	$\mathcal{O}(N)$
Sorted Array	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$
Linked List	$\mathcal{O}(N)$	$\mathcal{O}(1)$	$\mathcal{O}(N)$
Target (BST)	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$

$^{\mathcal{A}}$ Amortised time; linked list $\mathcal{O}(1)$ insert assumes a given node pointer.

The Solution Concept

Organise nodes into a **tree hierarchy** that halves the remaining search space at every step.

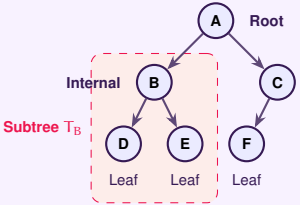
Trees: Definitions and Terminology

Mathematical Definition of a Tree

A **rooted tree** $T = (V, E)$ is a directed acyclic graph with a distinguished **root** vertex $r \in V$ of in-degree 0, where every other vertex $v \in V \setminus \{r\}$ has in-degree 1 and is reachable from r:

- **Root:** The unique vertex with in-degree 0.
- **Parent & Child:** A directed edge (u, v) indicates that u is the parent of v, and v is a child of u.
- **Leaf:** A node with out-degree 0 (no children).
- **Internal Node:** A node with out-degree $\geqslant 1$.
- **Subtree:** The tree induced by node u and all descendants reachable from u.

Tree Anatomy



Common Applications

File directories, HTML DOM trees, and Python abstract syntax trees (ASTs) are all rooted trees.

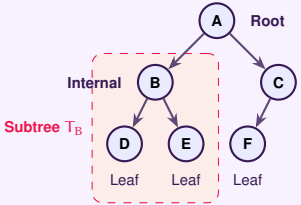
Trees: Definitions and Terminology

Mathematical Definition of a Tree

A **rooted tree** $T = (V, E)$ is a directed acyclic graph with a distinguished **root** vertex $r \in V$ of in-degree 0, where every other vertex $v \in V \setminus \{r\}$ has in-degree 1 and is reachable from r:

- **Root:** The unique vertex with in-degree 0.
- **Parent & Child:** A directed edge (u, v) indicates that u is the parent of v, and v is a child of u.
- **Leaf:** A node with out-degree 0 (no children).
- **Internal Node:** A node with out-degree $\geqslant 1$.
- **Subtree:** The tree induced by node u and all descendants reachable from u.

Tree Anatomy



Common Applications

File directories, HTML DOM trees, and Python abstract syntax trees (ASTs) are all rooted trees.

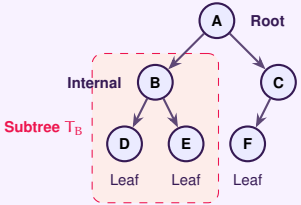
Trees: Definitions and Terminology

Mathematical Definition of a Tree

A **rooted tree** $T = (V, E)$ is a directed acyclic graph with a distinguished **root** vertex $r \in V$ of in-degree 0, where every other vertex $v \in V \setminus \{r\}$ has in-degree 1 and is reachable from r:

- **Root:** The unique vertex with in-degree 0.
- **Parent & Child:** A directed edge (u, v) indicates that u is the parent of v, and v is a child of u.
- **Leaf:** A node with out-degree 0 (no children).
- **Internal Node:** A node with out-degree $\geqslant 1$.
- **Subtree:** The tree induced by node u and all descendants reachable from u.

Tree Anatomy



Common Applications

File directories, HTML DOM trees, and Python abstract syntax trees (ASTs) are all rooted trees.

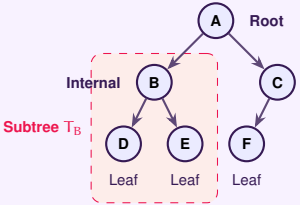
Trees: Definitions and Terminology

Mathematical Definition of a Tree

A **rooted tree** $T = (V, E)$ is a directed acyclic graph with a distinguished **root** vertex $r \in V$ of in-degree 0, where every other vertex $v \in V \setminus \{r\}$ has in-degree 1 and is reachable from r:

- **Root:** The unique vertex with in-degree 0.
- **Parent & Child:** A directed edge (u, v) indicates that u is the parent of v, and v is a child of u.
- **Leaf:** A node with out-degree 0 (no children).
- **Internal Node:** A node with out-degree $\geqslant 1$.
- **Subtree:** The tree induced by node u and all descendants reachable from u.

Tree Anatomy



Common Applications

File directories, HTML DOM trees, and Python abstract syntax trees (ASTs) are all rooted trees.

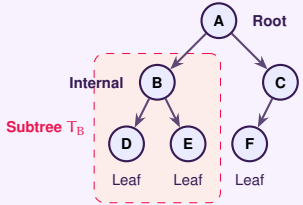
Trees: Definitions and Terminology

Mathematical Definition of a Tree

A **rooted tree** $T = (V, E)$ is a directed acyclic graph with a distinguished **root** vertex $r \in V$ of in-degree 0, where every other vertex $v \in V \setminus \{r\}$ has in-degree 1 and is reachable from r:

- **Root:** The unique vertex with in-degree 0.
- **Parent & Child:** A directed edge (u, v) indicates that u is the parent of v, and v is a child of u.
- **Leaf:** A node with out-degree 0 (no children).
- **Internal Node:** A node with out-degree $\geqslant 1$.
- **Subtree:** The tree induced by node u and all descendants reachable from u.

Tree Anatomy



Common Applications

File directories, HTML DOM trees, and Python abstract syntax trees (ASTs) are all rooted trees.

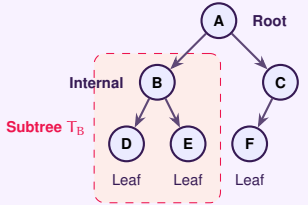
Trees: Definitions and Terminology

Mathematical Definition of a Tree

A **rooted tree** $T = (V, E)$ is a directed acyclic graph with a distinguished **root** vertex $r \in V$ of in-degree 0, where every other vertex $v \in V \setminus \{r\}$ has in-degree 1 and is reachable from r:

- **Root:** The unique vertex with in-degree 0.
- **Parent & Child:** A directed edge (u, v) indicates that u is the parent of v, and v is a child of u.
- **Leaf:** A node with out-degree 0 (no children).
- **Internal Node:** A node with out-degree $\geqslant 1$.
- **Subtree:** The tree induced by node u and all descendants reachable from u.

Tree Anatomy



Common Applications

File directories, HTML DOM trees, and Python abstract syntax trees (ASTs) are all rooted trees.

Tree Metrics: Depth, Height, and Binary Trees

Formal Definitions (Edge-Count)

- **Binary Tree:** At most two distinguished children per node (`left` and `right`).
- **Depth of Node u:** Number of edges on the path from root to u. $\mathrm{depth}(\mathrm{root}) = 0$.
- **Height of Tree H:** Maximum depth in T: $H = \max_{u \in T} \mathrm{depth}(u)$ ($H = 0$ for a single node; $H = -1$ for empty).

Height Bounds for $N \geqslant 1$ Nodes

Level d holds at most 2^d nodes, giving $N \leqslant 2^{H+1} - 1$,

$$\lfloor \log_2 N \rfloor \leqslant H \leqslant N - 1$$

Height Extremes for $N = 7$ Nodes

Algorithmic Consequence

Tree height H governs search comparisons. Goal: maintain $H = \mathcal{O}(\log N)$.

Tree Metrics: Depth, Height, and Binary Trees

Formal Definitions (Edge-Count)

- **Binary Tree:** At most two distinguished children per node (`left` and `right`).
- **Depth of Node u:** Number of edges on the path from root to u. $\text{depth}(\text{root}) = 0$.
- **Height of Tree H:** Maximum depth in T: $H = \max_{u \in T} \text{depth}(u)$ ($H = 0$ for a single node; $H = -1$ for empty).

Height Bounds for $N \geqslant 1$ Nodes

Level d holds at most 2^d nodes, giving $N \leqslant 2^{H+1} - 1$,

$$\lfloor \log_2 N \rfloor \leqslant H \leqslant N - 1$$

Height Extremes for $N = 7$ Nodes

Algorithmic Consequence

Tree height H governs search comparisons. Goal: maintain $H = \mathcal{O}(\log N)$.

Tree Metrics: Depth, Height, and Binary Trees

Formal Definitions (Edge-Count)

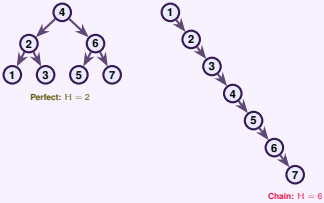
- **Binary Tree:** At most two distinguished children per node (`left` and `right`).
- **Depth of Node u:** Number of edges on the path from root to u. $\text{depth}(\text{root}) = 0$.
- **Height of Tree** H**:** Maximum depth in T: $H = \max_{u \in T} \text{depth}(u)$ ($H = 0$ for a single node; $H = -1$ for empty).

Height Bounds for $N \geqslant 1$ Nodes

Level d holds at most 2^d nodes, giving $N \leqslant 2^{H+1} - 1$,

$$\lfloor \log_2 N \rfloor \leqslant H \leqslant N - 1$$

Height Extremes for $N = 7$ Nodes



Algorithmic Consequence

Tree height H governs search comparisons. Goal: maintain $H = \mathcal{O}(\log N)$.

Tree Metrics: Depth, Height, and Binary Trees

Formal Definitions (Edge-Count)

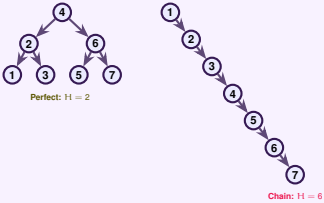
- **Binary Tree:** At most two distinguished children per node (`left` and `right`).
- **Depth of Node u:** Number of edges on the path from root to u. $\text{depth}(\text{root}) = 0$.
- **Height of Tree** H**:** Maximum depth in T: $H = \max_{u \in T} \text{depth}(u)$ ($H = 0$ for a single node; $H = -1$ for empty).

Height Bounds for $N \geqslant 1$ Nodes

Level d holds at most 2^d nodes, giving $N \leqslant 2^{H+1} - 1$,

$$\lfloor \log_2 N \rfloor \leqslant H \leqslant N - 1$$

Height Extremes for $N = 7$ Nodes



Algorithmic Consequence

Tree height H governs search comparisons. Goal: maintain $H = \mathcal{O}(\log N)$.

Binary Search Trees & Invariants

The Binary Search Tree (BST) Invariant

Formal BST Ordering Invariant

Assuming distinct keys drawn from a totally ordered set, a binary tree is a **Binary Search Tree** if and only if for **every** node X in the tree:

1. For all $u \in \text{LeftSubtree}(X)$: $u.\text{key} < X.\text{key}$
2. For all $v \in \text{RightSubtree}(X)$: $v.\text{key} > X.\text{key}$

Global Property

The condition applies to **all ancestors and descendants**, not merely immediate children.

Subtree Key Ranges

Consequence for Search

Comparing a search key against $X.\text{key}$ eliminates an entire subtree in $\mathcal{O}(1)$ time.

The Binary Search Tree (BST) Invariant

Formal BST Ordering Invariant

Assuming distinct keys drawn from a totally ordered set, a binary tree is a **Binary Search Tree** if and only if for **every** node X in the tree:

1. For all $u \in \text{LeftSubtree}(X)$: $u.\text{key} < X.\text{key}$
2. For all $v \in \text{RightSubtree}(X)$: $v.\text{key} > X.\text{key}$

Global Property

The condition applies to **all ancestors and descendants**, not merely immediate children.

Subtree Key Ranges

Consequence for Search

Comparing a search key against $X.\text{key}$ eliminates an entire subtree in $\mathcal{O}(1)$ time.

The Binary Search Tree (BST) Invariant

Formal BST Ordering Invariant

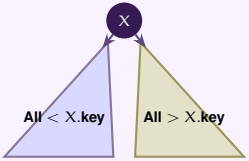
Assuming distinct keys drawn from a totally ordered set, a binary tree is a **Binary Search Tree** if and only if for **every** node X in the tree:

1. For all $u \in \text{LeftSubtree}(X)$: $u.\text{key} < X.\text{key}$
2. For all $v \in \text{RightSubtree}(X)$: $v.\text{key} > X.\text{key}$

Global Property

The condition applies to **all ancestors and descendants**, not merely immediate children.

Subtree Key Ranges



Consequence for Search

Comparing a search key against $X.\text{key}$ eliminates an entire subtree in $\mathcal{O}(1)$ time.

The Binary Search Tree (BST) Invariant

Formal BST Ordering Invariant

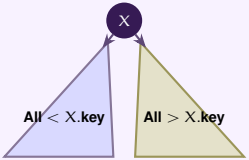
Assuming distinct keys drawn from a totally ordered set, a binary tree is a **Binary Search Tree** if and only if for **every** node X in the tree:

1. For all $u \in \text{LeftSubtree}(X)$: $u.\text{key} < X.\text{key}$
2. For all $v \in \text{RightSubtree}(X)$: $v.\text{key} > X.\text{key}$

Global Property

The condition applies to **all ancestors and descendants**, not merely immediate children.

Subtree Key Ranges



Consequence for Search

Comparing a search key against X.key eliminates an entire subtree in $\mathcal{O}(1)$ time.

Checkpoint 1: Spot the Invariant Violation

Inspect These Two Candidate Trees



The Diagnostic Question

In Tree B, node 15 satisfies $8 < 15 < 20$ locally. Is Tree B a valid BST? Why would `find(8)` fail?

Analysis

Tree B is invalid:

- Node 8 sits in the **right subtree of 10**.
- The invariant requires every node in 10's right subtree to be > 10. Since $8 < 10$, this violates the global invariant.

Search Failure Mode

When searching for 8:

1. At root 10: $8 < 10 \implies$ search branches **left**.
2. Node 8 sits on the right, so search never finds it.

Valid key range for 15's left child is $(10, 15)$.

Checkpoint 1: Spot the Invariant Violation

Inspect These Two Candidate Trees



The Diagnostic Question

In Tree B, node 15 satisfies $8 < 15 < 20$ locally. Is Tree B a valid BST? Why would `find(8)` fail?

Analysis

Tree B is invalid:

- Node 8 sits in the **right subtree of 10**.
- The invariant requires every node in 10's right subtree to be > 10. Since $8 < 10$, this violates the global invariant.

Search Failure Mode

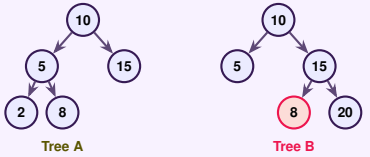
When searching for 8:

1. At root 10: $8 < 10 \implies$ search branches **left**.
2. Node 8 sits on the right, so search never finds it.

Valid key range for 15's left child is $(10, 15)$.

Checkpoint 1: Spot the Invariant Violation

Inspect These Two Candidate Trees



The Diagnostic Question

In Tree B, node 15 satisfies $8 < 15 < 20$ locally. Is Tree B a valid BST? Why would `find(8)` fail?

Analysis

Tree B is invalid:

- Node 8 sits in the **right subtree of 10**.
- The invariant requires every node in 10's right subtree to be > 10. Since $8 < 10$, this violates the global invariant.

Search Failure Mode

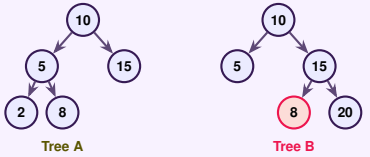
When searching for 8:

1. At root 10: $8 < 10 \implies$ search branches **left**.
2. Node 8 sits on the right, so search never finds it.

Valid key range for 15's left child is $(10, 15)$.

Checkpoint 1: Spot the Invariant Violation

Inspect These Two Candidate Trees



The Diagnostic Question

In Tree B, node 15 satisfies $8 < 15 < 20$ locally. Is Tree B a valid BST? Why would `find(8)` fail?

Analysis

Tree B is invalid:

- Node 8 sits in the **right subtree of 10**.
- The invariant requires every node in 10's right subtree to be > 10. Since $8 < 10$, this violates the global invariant.

Search Failure Mode

When searching for 8:

1. At root 10: $8 < 10 \implies$ search branches **left**.
2. Node 8 sits on the right, so search never finds it.

Valid key range for 15's left child is $(10, 15)$.

BST Representation: Tree and Node Classes

```python
class TreeNode:
  def __init__(self, key, value):
    self.key = key
    self.value = value
    self.left = None
    self.right = None

class BinaryTree:
  def __init__(self):
    self.root = None

  def lookup(self, key):
    if self.root is None:
      return None
    node = self.root.find(key)
    return node.value if node else None
```

Tree and Node Separation

Why use both `BinaryTree` and `TreeNode`?

- **Empty Trees:** When the tree contains no elements, `self.root = None` represents size 0 directly.
- **Root Reassignment:** When deleting the root node, `self.root` is reassigned directly.

Encapsulation

Callers call `tree.lookup(key)` on the wrapper without managing recursive node references directly.

BST Representation: Tree and Node Classes

```python
class TreeNode:
  def __init__(self, key, value):
    self.key = key
    self.value = value
    self.left = None
    self.right = None

class BinaryTree:
  def __init__(self):
    self.root = None

  def lookup(self, key):
    if self.root is None:
      return None
    node = self.root.find(key)
    return node.value if node else None
```

Tree and Node Separation

Why use both `BinaryTree` and `TreeNode`?

- **Empty Trees:** When the tree contains no elements, `self.root = None` represents size 0 directly.
- **Root Reassignment:** When deleting the root node, `self.root` is reassigned directly.

Encapsulation

Callers call `tree.lookup(key)` on the wrapper without managing recursive node references directly.

BST Lookup (`find`): Halving the Search Space

```
1  class TreeNode:
2    def find(self, key):
3      if key == self.key:
4        return self
5      if key < self.key:
6        if self.left is None:
7          return None
8        return self.left.find(key)
9      else:
10       if self.right is None:
11         return None
12       return self.right.find(key)
```

Trace: `find(12)`

Step-by-Step

1. At 10: $12 > 10 \implies$ branch right.
2. At 15: $12 < 15 \implies$ branch left.
3. At 12: $12 == 12 \implies$ match returned.

3 comparisons instead of 5 for a linear list.

Complexity Derivation

Each step descends one level, visiting at most $H + 1$ nodes,

$$T(N) = \mathcal{O}(H)$$

Balanced tree: $\mathcal{O}(\log N)$. Degenerate chain: $\mathcal{O}(N)$.

BST Lookup (`find`): Halving the Search Space

```
1  class TreeNode:
2    def find(self, key):
3      if key == self.key:
4        return self
5      if key < self.key:
6        if self.left is None:
7          return None
8        return self.left.find(key)
9      else:
10       if self.right is None:
11         return None
12       return self.right.find(key)
```

Complexity Derivation

Each step descends one level, visiting at most $H + 1$ nodes,

$$T(N) = \mathcal{O}(H)$$

Balanced tree: $\mathcal{O}(\log N)$. Degenerate chain: $\mathcal{O}(N)$.

Trace: `find(12)`

Step-by-Step

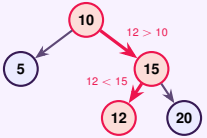
1. At 10: $12 > 10 \implies$ branch right.
2. At 15: $12 < 15 \implies$ branch left.
3. At 12: 12 == 12 $\implies$ match returned.

3 comparisons instead of 5 for a linear list.

BST Lookup (`find`): Halving the Search Space

```
1  class TreeNode:
2    def find(self, key):
3      if key == self.key:
4        return self
5      if key < self.key:
6        if self.left is None:
7          return None
8        return self.left.find(key)
9      else:
10       if self.right is None:
11         return None
12       return self.right.find(key)
```

Trace: `find(12)`



Step-by-Step

1. At 10: $12 > 10 \implies$ branch right.
2. At 15: $12 < 15 \implies$ branch left.
3. At 12: 12 == 12 $\implies$ match returned.

3 comparisons instead of 5 for a linear list.

Complexity Derivation

Each step descends one level, visiting at most $H + 1$ nodes,

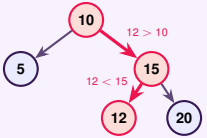
$$T(N) = \mathcal{O}(H)$$

Balanced tree: $\mathcal{O}(\log N)$. Degenerate chain: $\mathcal{O}(N)$.

BST Lookup (`find`): Halving the Search Space

```
class TreeNode:
  def find(self, key):
    if key == self.key:
      return self
    if key < self.key:
      if self.left is None:
        return None
      return self.left.find(key)
    else:
      if self.right is None:
        return None
      return self.right.find(key)
```

Trace: `find(12)`



Step-by-Step

1. At 10: $12 > 10 \implies$ branch right.
2. At 15: $12 < 15 \implies$ branch left.
3. At 12: `12 == 12` $\implies$ match returned.

3 comparisons instead of 5 for a linear list.

Complexity Derivation

Each step descends one level, visiting at most $H + 1$ nodes,

$$T(N) = \mathcal{O}(H)$$

Balanced tree: $\mathcal{O}(\log N)$. Degenerate chain: $\mathcal{O}(N)$.

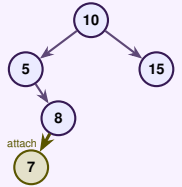
BST Insertion (`insert`): Growing at the Leaves

```python
class TreeNode:
  def insert(self, key, value):
    if key < self.key:
      if self.left is None:
        self.left = TreeNode(key, value)
      else:
        self.left.insert(key, value)
    else:
      if self.right is None:
        self.right = TreeNode(key, value)
      else:
        self.right.insert(key, value)
```

Insertion Mechanics

`insert` follows the path of a failed `find`. At `None`, it attaches a new leaf in $\mathcal{O}(1)$ pointer updates. Total time: $\mathcal{O}(H)$.

Trace: `insert(7, "G")`



Key Observation

Unlike dynamic arrays, inserting into a BST does not require shifting existing nodes in memory.

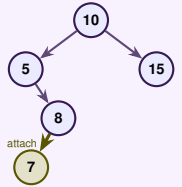
BST Insertion (`insert`): Growing at the Leaves

```python
class TreeNode:
  def insert(self, key, value):
    if key < self.key:
      if self.left is None:
        self.left = TreeNode(key, value)
      else:
        self.left.insert(key, value)
    else:
      if self.right is None:
        self.right = TreeNode(key, value)
      else:
        self.right.insert(key, value)
```

Insertion Mechanics

`insert` follows the path of a failed `find`. At `None`, it attaches a new leaf in $\mathcal{O}(1)$ pointer updates. Total time: $\mathcal{O}(H)$.

Trace: `insert(7, "G")`



Key Observation

Unlike dynamic arrays, inserting into a BST does not require shifting existing nodes in memory.

Handling Duplicate Keys in `tree.py`

The Implementation in `tree.py`

In `tree.py`, line 44 writes:

```
1 if key < self.key:
2   ...
3 else:
4   # Routes key >= self.key to right:
5   if self.right is None:
6     self.right = TreeNode(key, value)
7   else:
8     self.right.insert(key, value)
```

If `key == self.key`, it creates a duplicate node in the right subtree.

Effect of Duplicate Keys

Because `find` stops at the first matching key:

- Re-inserting an existing key shadows the new value at a leaf below.
- `lookup` will never reach the updated value.

The Map ADT Correction

In a standard key-value map, matching keys update the value in-place:

```
1 elif key == self.key:
2   self.value = value
```

Preserves uniqueness and updates in $\mathcal{O}(H)$ time.

Handling Duplicate Keys in `tree.py`

The Implementation in `tree.py`

In `tree.py`, line 44 writes:

```
1 if key < self.key:
2   ...
3 else:
4   # Routes key >= self.key to right:
5   if self.right is None:
6     self.right = TreeNode(key, value)
7   else:
8     self.right.insert(key, value)
```

If `key == self.key`, it creates a duplicate node in the right subtree.

Effect of Duplicate Keys

Because `find` stops at the first matching key:

- Re-inserting an existing key shadows the new value at a leaf below.
- `lookup` will never reach the updated value.

The Map ADT Correction

In a standard key-value map, matching keys update the value in-place:

```
1 elif key == self.key:
2   self.value = value
```

Preserves uniqueness and updates in $\mathcal{O}(\mathrm{H})$ time.

Tree Traversals & Python Generators

Depth-First Tree Traversals

Three Classic Depth-First Orders

- **Pre-Order ($\mathbf{N} \to \mathbf{L} \to \mathbf{R}$):** Visit node before subtrees. Used for cloning or serialising tree structure.
- **In-Order ($\mathbf{L} \to \mathbf{N} \to \mathbf{R}$):** Visit left subtree, then node, then right subtree.
- **Post-Order ($\mathbf{L} \to \mathbf{R} \to \mathbf{N}$):** Visit subtrees before node. Used for bottom-up computation (e.g. subtree size).

In-Order Traversal Theorem

An in-order traversal of a BST with distinct keys visits keys in **strictly increasing sorted order**.

Sorted Order via Projection

Visual Intuition

Projecting every node vertically onto a horizontal line reproduces the sorted sequence directly.

Depth-First Tree Traversals

Three Classic Depth-First Orders

- **Pre-Order ($\mathbf{N} \to \mathbf{L} \to \mathbf{R}$):** Visit node before subtrees. Used for cloning or serialising tree structure.
- **In-Order ($\mathbf{L} \to \mathbf{N} \to \mathbf{R}$):** Visit left subtree, then node, then right subtree.
- **Post-Order ($\mathbf{L} \to \mathbf{R} \to \mathbf{N}$):** Visit subtrees before node. Used for bottom-up computation (e.g. subtree size).

In-Order Traversal Theorem

An in-order traversal of a BST with distinct keys visits keys in **strictly increasing sorted order**.

Sorted Order via Projection

Visual Intuition

Projecting every node vertically onto a horizontal line reproduces the sorted sequence directly.

Depth-First Tree Traversals

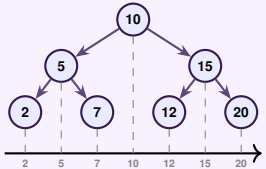
Three Classic Depth-First Orders

- **Pre-Order ($\mathbf{N} \to \mathbf{L} \to \mathbf{R}$):** Visit node before subtrees. Used for cloning or serialising tree structure.
- **In-Order ($\mathbf{L} \to \mathbf{N} \to \mathbf{R}$):** Visit left subtree, then node, then right subtree.
- **Post-Order ($\mathbf{L} \to \mathbf{R} \to \mathbf{N}$):** Visit subtrees before node. Used for bottom-up computation (e.g. subtree size).

In-Order Traversal Theorem

An in-order traversal of a BST with distinct keys visits keys in **strictly increasing sorted order**.

Sorted Order via Projection



Visual Intuition

Projecting every node vertically onto a horizontal line reproduces the sorted sequence directly.

Depth-First Tree Traversals

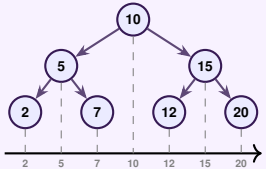
Three Classic Depth-First Orders

- **Pre-Order ($\mathbf{N} \to \mathbf{L} \to \mathbf{R}$):** Visit node before subtrees. Used for cloning or serialising tree structure.
- **In-Order ($\mathbf{L} \to \mathbf{N} \to \mathbf{R}$):** Visit left subtree, then node, then right subtree.
- **Post-Order ($\mathbf{L} \to \mathbf{R} \to \mathbf{N}$):** Visit subtrees before node. Used for bottom-up computation (e.g. subtree size).

In-Order Traversal Theorem

An in-order traversal of a BST with distinct keys visits keys in **strictly increasing sorted order**.

Sorted Order via Projection



Visual Intuition

Projecting every node vertically onto a horizontal line reproduces the sorted sequence directly.

Limitations of Returning a List

Implementation: Returning a List

```
1 def walk_list(self):
2   result = []
3   if self.left is not None:
4     result.extend(self.left.walk_list())
5   result.append((self.key, self.value))
6   if self.right is not None:
7     result.extend(self.right.walk_list())
8   return result
```

Three Drawbacks

1. **Memory Overhead ($\Theta(N)$):** Allocates a list holding all N elements simultaneously.
2. **Initial Latency:** The caller cannot inspect element 1 until all N nodes have been collected.
3. **Early Exit Cost:** If searching for the first matching item, building the remainder of the list is redundant.

The Alternative: Lazy Evaluation

Yield individual items on demand without allocating an intermediate container.

Limitations of Returning a List

Implementation: Returning a List

```
1 def walk_list(self):
2   result = []
3   if self.left is not None:
4     result.extend(self.left.walk_list())
5   result.append((self.key, self.value))
6   if self.right is not None:
7     result.extend(self.right.walk_list())
8   return result
```

Three Drawbacks

1. **Memory Overhead ($\Theta(N)$):** Allocates a list holding all N elements simultaneously.
2. **Initial Latency:** The caller cannot inspect element 1 until all N nodes have been collected.
3. **Early Exit Cost:** If searching for the first matching item, building the remainder of the list is redundant.

The Alternative: Lazy Evaluation

Yield individual items on demand without allocating an intermediate container.

Python Generators and `yield` (`yield-example-1.py`)

```python
def return_example():
  return [1, 2, 3]  # Function terminates

def yield_example():
  yield 1  # Suspends execution
  yield 2  # Resumes here on next()
  yield 3

gen = yield_example()
print(next(gen))  # 1
print(next(gen))  # 2
```

`return` vs `yield`

- **`return`:** Cleans up local function variables and returns control permanently.
- **`yield`:** Suspends the generator frame on the heap, preserves local variables, and produces a value.

The Iterator Protocol

Calling a generator function returns a **generator iterator**. A `for` loop calls `next()` repeatedly until `StopIteration` is raised.

Python Generators and `yield` (`yield-example-1.py`)

```python
def return_example():
  return [1, 2, 3]  # Function terminates

def yield_example():
  yield 1  # Suspends execution
  yield 2  # Resumes here on next()
  yield 3

gen = yield_example()
print(next(gen))  # 1
print(next(gen))  # 2
```

`return` vs `yield`

- **`return`:** Cleans up local function variables and returns control permanently.
- **`yield`:** Suspends the generator frame on the heap, preserves local variables, and produces a value.

The Iterator Protocol

Calling a generator function returns a **generator iterator**. A `for` loop calls `next()` repeatedly until `StopIteration` is raised.



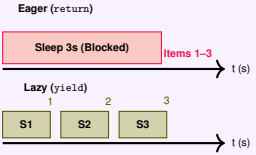
Lazy vs Eager Evaluation (`yield-example-2.py`)

```python
import time

def return_example():
  result = []
  for i in range(1, 4):
    time.sleep(1) # simulate work
    result.append(i)
  return result

def yield_example():
  for i in range(1, 4):
    time.sleep(1) # simulate work
    yield i
```

Execution Timeline Comparison



Interleaved Execution

Time-to-first-item: 1 s with `yield` vs 3 s with `return`. Execution is single-threaded and interleaved (cooperative suspension and resumption).

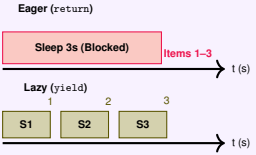
Lazy vs Eager Evaluation (`yield-example-2.py`)

```python
import time

def return_example():
  result = []
  for i in range(1, 4):
    time.sleep(1) # simulate work
    result.append(i)
  return result

def yield_example():
  for i in range(1, 4):
    time.sleep(1) # simulate work
    yield i
```

Execution Timeline Comparison



Interleaved Execution

Time-to-first-item: 1 s with `yield` vs 3 s with `return`. Execution is single-threaded and interleaved (cooperative suspension and resumption).

Recursive Generator Delegation: `yield from`

Manual Iteration Delegation

```
# Iterating over a sub-generator:
for item in subgenerator():
    yield item
```

Direct Delegation: PEP 380

```
# Direct delegation:
yield from subgenerator()
```

How `yield from` Operates

`yield from <iterable>` delegates iteration directly to a sub-generator:

- Values produced by the sub-generator are passed directly to the outer caller.
- Sub-generator termination is handled automatically without manual loop boilerplate.

Recursive Traversals

Enables writing recursive tree traversals directly in clean Python.

Recursive Generator Delegation: `yield from`

Manual Iteration Delegation

```
1 # Iterating over a sub-generator:
2 for item in subgenerator():
3   yield item
```

Direct Delegation: PEP 380

```
1 # Direct delegation:
2 yield from subgenerator()
```

How `yield from` Operates

`yield from <iterable>` delegates iteration directly to a sub-generator:

- Values produced by the sub-generator are passed directly to the outer caller.
- Sub-generator termination is handled automatically without manual loop boilerplate.

Recursive Traversals

Enables writing recursive tree traversals directly in clean Python.



In-Order BST Generator (`TreeNode.walk`)

```python
class TreeNode:
  def walk(self):
    if self.left is not None:
      yield from self.left.walk()
    yield (self.key, self.value)
    if self.right is not None:
      yield from self.right.walk()

# Consuming the generator:
for key, val in tree.root.walk():
  print(key, val)
```

Time and Space Complexity

- **Total Visits:** $\Theta(N)$ node visits in total.
- **First Item Latency:** $\mathcal{O}(H)$ to walk down to the minimum key.
- **Auxiliary Memory:** $\Theta(H)$ suspended generator frames on the heap along the active path ($\Theta(\log N)$ balanced, $\Theta(N)$ degenerate).

Heap Memory Savings

In a balanced BST of 10^6 nodes, `walk()` maintains ≈ 20 suspended frames, compared to an eager list holding 10^6 tuples.

In-Order BST Generator (`TreeNode.walk`)

```python
class TreeNode:
  def walk(self):
    if self.left is not None:
      yield from self.left.walk()
    yield (self.key, self.value)
    if self.right is not None:
      yield from self.right.walk()

# Consuming the generator:
for key, val in tree.root.walk():
  print(key, val)
```

Time and Space Complexity

- **Total Visits:** $\Theta(N)$ node visits in total.
- **First Item Latency:** $\mathcal{O}(H)$ to walk down to the minimum key.
- **Auxiliary Memory:** $\Theta(H)$ suspended generator frames on the heap along the active path ($\Theta(\log N)$ balanced, $\Theta(N)$ degenerate).

Heap Memory Savings

In a balanced BST of 10^6 nodes, `walk()` maintains ≈ 20 suspended frames, compared to an eager list holding 10^6 tuples.

In-Order BST Generator (`TreeNode.walk`)

```python
class TreeNode:
  def walk(self):
    if self.left is not None:
      yield from self.left.walk()
    yield (self.key, self.value)
    if self.right is not None:
      yield from self.right.walk()

# Consuming the generator:
for key, val in tree.root.walk():
  print(key, val)
```

Time and Space Complexity

- **Total Visits:** $\Theta(N)$ node visits in total.
- **First Item Latency:** $\mathcal{O}(H)$ to walk down to the minimum key.
- **Auxiliary Memory:** $\boldsymbol{\Theta(H)}$ suspended generator frames on the heap along the active path ($\Theta(\log N)$ balanced, $\Theta(N)$ degenerate).

Heap Memory Savings

In a balanced BST of 10^6 nodes, `walk()` maintains ≈ 20 suspended frames, compared to an eager list holding 10^6 tuples.

In-Order BST Generator (`TreeNode.walk`)

```python
class TreeNode:
  def walk(self):
    if self.left is not None:
      yield from self.left.walk()
    yield (self.key, self.value)
    if self.right is not None:
      yield from self.right.walk()

# Consuming the generator:
for key, val in tree.root.walk():
  print(key, val)
```

Time and Space Complexity

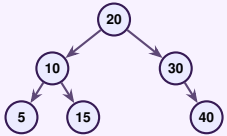
- **Total Visits:** $\Theta(N)$ node visits in total.
- **First Item Latency:** $\mathcal{O}(H)$ to walk down to the minimum key.
- **Auxiliary Memory:** $\boldsymbol{\Theta(H)}$ suspended generator frames on the heap along the active path ($\Theta(\log N)$ balanced, $\Theta(N)$ degenerate).

Heap Memory Savings

In a balanced BST of 10^6 nodes, `walk()` maintains ≈ 20 suspended frames, compared to an eager list holding 10^6 tuples.

Checkpoint 2: Tree Traversals and Generators

Example Tree



Question 1: Lazy Generators

When running `g = tree.root.walk()`, how many nodes are visited before we call `next(g)`?

Question 2: Traversal Orders

In what order are the keys visited?

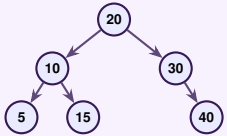
1. **Pre-order (Node, L, R):** 20, 10, 5, 15, 30, 40
2. **In-order (L, Node, R):** 5, 10, 15, 20, 30, 40 (sorted)
3. **Post-order (L, R, Node):** 5, 15, 10, 40, 30, 20

Answer to Question 1

Zero nodes. Calling `walk()` only creates the generator object. Python runs no code inside the function until we request the first item with `next()` or a `for` loop.

Checkpoint 2: Tree Traversals and Generators

Example Tree



Question 1: Lazy Generators

When running `g = tree.root.walk()`, how many nodes are visited before we call `next(g)`?

Question 2: Traversal Orders

In what order are the keys visited?

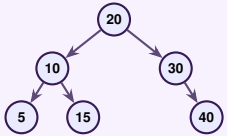
1. **Pre-order (Node, L, R):** 20, 10, 5, 15, 30, 40
2. **In-order (L, Node, R):** 5, 10, 15, 20, 30, 40 (sorted)
3. **Post-order (L, R, Node):** 5, 15, 10, 40, 30, 20

Answer to Question 1

Zero nodes. Calling `walk()` only creates the generator object. Python runs no code inside the function until we request the first item with `next()` or a `for` loop.

Checkpoint 2: Tree Traversals and Generators

Example Tree



Question 1: Lazy Generators

When running `g = tree.root.walk()`, how many nodes are visited before we call `next(g)`?

Question 2: Traversal Orders

In what order are the keys visited?

1. **Pre-order (Node, L, R):** 20, 10, 5, 15, 30, 40
2. **In-order (L, Node, R):** 5, 10, 15, 20, 30, 40 (sorted)
3. **Post-order (L, R, Node):** 5, 15, 10, 40, 30, 20

Answer to Question 1

Zero nodes. Calling `walk()` only creates the generator object. Python runs no code inside the function until we request the first item with `next()` or a `for` loop.

Binary Search Tree Deletion

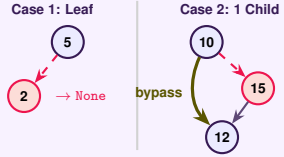
BST Deletion: Recursive Pattern and Cases 1 and 2

```python
# Inside TreeNode.delete(self, key):
if key < self.key and self.left:
  self.left = self.left.delete(key)
elif key > self.key and self.right:
  self.right = self.right.delete(key)
elif key == self.key:
  # Case 1: Leaf (no children)
  if self.left is None and self.right is None:
    return None
  # Case 2: Only one child
  if self.left is None:
    return self.right
  if self.right is None:
    return self.left
  # Case 3: Two children (next slide)
return self
```

Recursive Update Pattern

Each recursive call returns the updated child reference, such as `self.left = self.left.delete(key)`, so the parent updates its pointer without needing a parent link.

Cases 1 and 2



Deletion Case 3: A Node with Two Children

The Problem

When node D has two children, neither child can replace D directly without detaching the other branch.

Who Can Take Its Place?

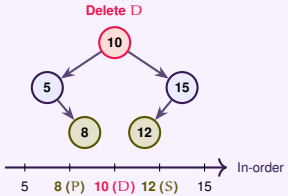
To keep the BST property, the replacement key R must satisfy

$$x.\text{key} < R < y.\text{key}$$

for all $x \in T_L \setminus \{R\}$ and $y \in T_R \setminus \{R\}$.
Only two keys qualify:

1. **In-order Predecessor:** largest key in left subtree ($P = \max(T_L)$).
2. **In-order Successor:** smallest key in right subtree ($S = \min(T_R)$).

Locating Predecessor and Successor



Key Insight: P Has No Right Child

Because P is the largest key on the left, it has no right child. Deleting P from T_L always reduces to Case 1 (leaf) or Case 2 (left child only).

Implementing Case 3: Copying Predecessor Data

```python
# Inside TreeNode.delete, Case 3:
else:
  # 1. Find predecessor in left subtree
  maxkey = self.left.maxkey()
  maxnode = self.left.find(maxkey)

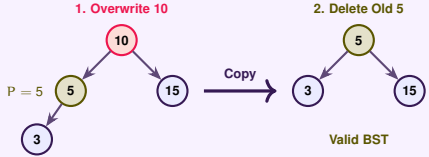
  # 2. Overwrite this node's data
  self.key = maxnode.key
  self.value = maxnode.value

  # 3. Delete predecessor from left
  self.left = self.left.delete(maxkey)
  return self
```

Why Copy Instead of Rewiring?

Overwriting the key and value avoids rewiring child pointers. We then delete the old predecessor node from T_L via Case 1 or Case 2.

Tree State Before and After



Base Case Guarantee and $\mathcal{O}(H)$ Time

P has no right child, so deleting it never triggers Case 3 again. Since `maxkey()`, `find()`, and `delete()` walk down at most tree height H, deletion runs in $\mathcal{O}(H)$ time.

BST Complexity and Tree Balance

BST Performance and The Sorted-Input Problem

Operation Complexity

Operation	Balanced Tree	Degenerate Tree
`lookup(k)`	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$
`insert(k, v)`	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$
`delete(k)`	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$
`walk()` (all keys)	$\Theta(N)$	$\Theta(N)$

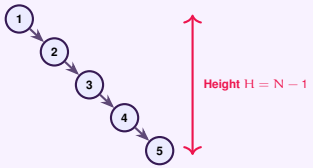
Tree Height Governs Time

Every search, insertion, and deletion follows a single root-to-leaf path. The work is proportional to the tree height H,

$$T(N) = \mathcal{O}(H).$$

What If Keys Arrive Sorted?

Inserting $[1, 2, 3, 4, 5]$ into an empty BST yields a chain:



Degenerate Search and $\Theta(N^2)$ Build Cost

Single searches degrade to $\mathcal{O}(N)$, and inserting N sorted keys one-by-one takes $1 + 2 + \cdots + N = \Theta(N^2)$ total comparisons.

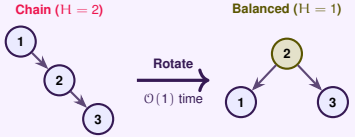
Self-Balancing Trees and Hash Motivation

Preventing the Worst Case

Standard BSTs depend on insertion order. In real systems, we prevent degenerate trees using **self-balancing trees** (such as AVL trees and Red-Black trees).

- During insert and delete, the tree checks branch heights.
- If one side becomes too tall, it performs a local **tree rotation** in $\mathcal{O}(1)$ time.
- This guarantees $\mathrm{H} = \mathcal{O}(\log \mathrm{N})$ in the worst case for all operations.

Example: Tree Rotation



Can We Do Better Than $\mathcal{O}(\log \mathrm{N})$?

Any comparison-based search needs $\Omega(\log \mathrm{N})$ comparisons in the worst case. To search faster, we stop comparing keys and compute array indices directly with **Hash Tables**.

Hash Tables and Collision Resolution

From Direct Addressing to Hash Functions

Direct Addressing

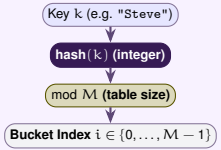
If keys are small integers $0 \leqslant k < M$, we can store items in an array `A[k]`.

- Lookup, insert, and delete take $\mathcal{O}(1)$ time.
- **Problem:** Real keys (strings, large IDs) come from a vast universe of possible values. We cannot create an array of size 2^{64}.

The Pigeonhole Principle

Because there are far more possible keys than table slots, different keys will sometimes produce the exact same index. This is called a **collision**.

The Hashing Pipeline



Resolving Collisions

- **Separate Chaining:** each bucket stores a list of items.
- **Open Addressing:** probe for the next empty slot.

Separate Chaining Implementation (`hashtable.py`)

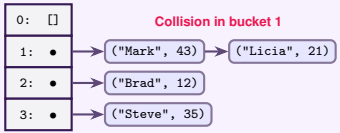
```python
class HashTable:
  def __init__(self, size=4):
    self.bucket_count = size
    self.buckets = [[] for _ in range(size)]

  def get_index(self, key):
    return hash(key) % self.bucket_count

  def insert(self, key, value):
    idx = self.get_index(key)
    self.buckets[idx].append((key, value))

  def lookup(self, key):
    idx = self.get_index(key)
    for k, v in self.buckets[idx]:
      if k == key:
        return v
    return None
```

Memory Model ($M = 4$)



Store Keys with Values

Both `"Mark"` and `"Licia"` hash to bucket 1. Storing `(key, value)` pairs lets `lookup` check `k == key` to return the right value.

Load Factor (α), Expected $\mathcal{O}(1)$ Time, and Resizing

The Load Factor (α)

For a table holding N items across M buckets, the **load factor** is

$$\alpha = \frac{\mathrm{N}}{\mathrm{M}}.$$

α represents the average number of items per bucket.

Expected Search Time

If keys are spread evenly across buckets:

- Computing bucket index takes $\mathcal{O}(1)$ time.
- Scanning bucket chain takes on average α steps.
- Total expected time is $\Theta(1 + \alpha)$.

When α is kept small, search is $\mathcal{O}(1)$ **on average**.

Dynamic Resizing (Rehashing)

When α passes a threshold (e.g. 0.75):

1. Allocate a table with double the buckets ($\mathrm{M} \to 2\mathrm{M}$).
2. Rehash each key: $\mathrm{hash}(\mathrm{k}) \bmod (2\mathrm{M})$.
3. Insert elements into new buckets.

Amortised Cost: Rehashing takes $\mathcal{O}(\mathrm{N})$ time, but happens rarely. Just like list growth in Week 2, insertion is **amortised** $\mathcal{O}(1)$.

Hashability and Assignment 4: Sets of Coordinates

The Hashability Rule

In Python, keys in a `dict` and items in a `set` must be **immutable**.

```
# Valid: tuples are immutable
coords = (5, 22)
visited = set()
visited.add(coords)  # Works

# Invalid: lists are mutable
bad = [5, 22]
# TypeError: unhashable type: 'list'
```

Why Must Keys Be Immutable?

If a key changed after insertion, its hash would change and the item would be lost in the wrong bucket.

Assignment 4: The Tetris Board

In the Tetris autoplayer, landed blocks are stored in `board.cells`:

- Python `set` of `(x, y)` coordinate tuples.
- `(x, y) in board.cells` takes $\mathcal{O}(1)$ average time.
- Sets have no guaranteed order. Scan rows and columns explicitly:

```
for y in range(board.height):
  for x in range(board.width):
    if (x, y) in board.cells:
      # Found occupied cell
```

Summary and Outlook

Data Structures Summary (Weeks 2 to 4) and Next Steps

Data Structure	Search	Insert	Delete	Ordered By
Python List (Unsorted)	$\mathcal{O}(N)$	$\mathcal{O}(1)$ amortised	$\mathcal{O}(N)$	Insertion order
Sorted List	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$	Sorted by key
Linked Deque (`deque`)	$\mathcal{O}(N)$	$\mathcal{O}(1)$ at ends	$\mathcal{O}(1)$ at ends	Insertion order
Balanced BST	$\mathcal{O}(\log \mathbf{N})$	$\mathcal{O}(\log \mathbf{N})$	$\mathcal{O}(\log \mathbf{N})$	Sorted by key
Degenerate BST (Worst)	$\mathcal{O}(N)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$	Sorted by key
Hash Table (`dict` / `set`)	$\mathcal{O}(\mathbf{1})$ avg	$\mathcal{O}(\mathbf{1})$ avg	$\mathcal{O}(\mathbf{1})$ avg	Unordered

Core Lessons from Week 4

- **BST Invariant:** Keeps keys sorted and gives $\mathcal{O}(\log N)$ search when balanced.
- **Generators (`yield`):** Stream data on demand with $\Theta(H)$ memory.
- **Hash Tables:** Map keys directly to buckets for $\mathcal{O}(1)$ average lookup.

Connecting to Week 5 & Assignment 4

- **Assignment 4:** Inspect `board.cells` (a hash `set` of coordinate tuples).
- **Week 5:** Explore mazes, depth-first search, and the Minimax game algorithm.